Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Вдосконалення структури коду графічного редактора об’єктів»

Виконав:

студент групи ІМ-о51
Мосендз Белла Яківна
Номер в списку - 9

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab4.

2. Написати вихідний текст програми згідно варіанту завдання.

3. Скомпілювати вихідний текст і отримати виконуваний файл програми.

4. Перевірити роботу програми. Налагодити програму.

5. Проаналізувати та прокоментувати результати та вихідний текст програми.

6. Оформити звіт.

Завдання згідно варіанту:

Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor.

3. Усі кольори та стилі (за винятком "гумового" сліду) геометричних форм – як у попередній лабораторній роботі №3. "Гумовий" слід при вводі усіх фігур малювати пунктирною лінією.

4. Окрім чотирьох типів фігур, які були у попередніх лаб. №2 та 3, запрограмувати ще введення та відображення двох нових фігур – лінія з кружечками та каркас куба. Кольори ліній та заповнення цих нових фігур студент визначає на свій розсуд. Для об’єктів типів лінії з кружечками та каркасу кубу відповідні класи запрограмувати саме множинним успадкуванням. Втім, якщо мова програмування не дозволяє множинне успадкування, то його можна замінити іншою конструкцією (наприклад, композицією). У першу чергу це стосується метода Show для нових фігур – для відображення ліній треба використовувати виклики метода Show з класу LineShape, для відображення кружечків – виклики метода Show з класу EllipseShape, а для відображення прямокутників – виклики метода Show з класу RectShape.

5. Для усіх шести типів форм зробити кнопки Toolbar з підказками (tooltips)

6. У звіті повинна бути схема успадкування класів – діаграма класів. Для побудови діаграми класів дозволяється обрати інструменти на власний розсуд

Main file: MainWindow.xaml.cs

```
using System.Windows;

using System.Windows.Input;

using System.Windows.Controls;



namespace Lab4;



public partial class MainWindow : Window

{

    public MainWindow()

    {

        InitializeComponent();

        DrawToolbarIcons();

        App.Editor.AttachCanvas(drawingCanvas);

        SelectTool(DrawingTool.Point);

        UpdateShapeCount();

    }



    private void SelectTool(DrawingTool tool)

    {

        App.Editor.SelectTool(tool);
```

```
        pointMenuItem.IsChecked =
            tool == DrawingTool.Point;

        lineMenuItem.IsChecked =
            tool == DrawingTool.Line;

        rectangleMenuItem.IsChecked =
            tool == DrawingTool.Rectangle;

        ellipseMenuItem.IsChecked =
            tool == DrawingTool.Ellipse;

        lineWithCirclesMenuItem.IsChecked =
            tool == DrawingTool.LineWithCircles;

        cubeMenuItem.IsChecked =
            tool == DrawingTool.Cube;
    }

    private void OnNotify(object sender, RoutedEventArgs e)
    {
```

```
        FrameworkElement element = (FrameworkElement)sender;
        string toolName = (string)element.Tag;
        DrawingTool tool = Enum.Parse<DrawingTool>(toolName);

        SelectTool(tool);
    }

    // масив кортежів (tuple)
    private void DrawToolbarIcons()
    {
        var icons = new (Canvas canvas, DrawingTool tool, Point start,
Point end)[]
        {
        (pointIconCanvas, DrawingTool.Point,
            new Point(20, 16), new Point(20, 16)),

        (lineIconCanvas, DrawingTool.Line,
            new Point(6, 24), new Point(34, 8)),

        (rectangleIconCanvas, DrawingTool.Rectangle,
            new Point(8, 8), new Point(32, 24)),
```

```
        (ellipseIconCanvas, DrawingTool.Ellipse,
            new Point(20, 16), new Point(32, 24)),

        (lineWithCirclesIconCanvas, DrawingTool.LineWithCircles,
            new Point(11, 16), new Point(29, 16)),

        (cubeIconCanvas, DrawingTool.Cube,
            new Point(8, 12), new Point(26, 30))
        };

        foreach (var (canvas, tool, start, end) in icons)
        {
            canvas.Children.Add(
                ShapeFactory.Create(tool, start, end).Show(false)
            );
        }
    }

    private void DrawingCanvas_MouseLeftButtonDown(object sender,
MouseButtonEventArgs e)
    {
        Point position = e.GetPosition(drawingCanvas);
```

```
        bool success = App.Editor.BeginDrawing(position);



        if (!success)

        {

            MessageBox.Show(

                $"Досягнуто максимальної кількості фігур
({MyEditor.MaxShapes})."

            );

        }



        UpdateShapeCount();

    }



    private void DrawingCanvas_MouseMove(object sender, MouseEventArgs e)

    {

        Point position = e.GetPosition(drawingCanvas);

        App.Editor.UpdateDrawing(position);

    }



    private void DrawingCanvas_MouseLeftButtonUp(object sender,
MouseButtonEventArgs e)

    {
```

```
        Point position = e.GetPosition(drawingCanvas);

        App.Editor.EndDrawing(position);



        UpdateShapeCount();

    }



    private void UpdateShapeCount()

    {

        shapeCountText.Text = $"Фігур: {App.Editor.ShapeCount} /
{MyEditor.MaxShapes}";

    }

}
```

DrawingTools.cs

```
namespace Lab4;



public enum DrawingTool

{

    Point,

    Line,

    Rectangle,

    Ellipse,
```

```
    LineWithCircles,

    Cube

}
```

App.xaml.cs

```
using System.Windows;



namespace Lab4;



public partial class App : Application

{

    internal static readonly MyEditor Editor = new MyEditor();

}
```

MyEditor.cs

```
using System.Windows;

using System.Windows.Controls;

using Lab4.Models;



namespace Lab4;
```

```
internal sealed class MyEditor

{

    internal const int MaxShapes = 110;

    private readonly Shape?[] _shapes = new Shape?[MaxShapes];

    private int _shapeCount = 0;

    private DrawingTool _currentTool = DrawingTool.Point;

    private Point? _dragStart;

    private UIElement? _previewShape;

    private Canvas? _canvas;



    internal int ShapeCount => _shapeCount;



    internal void AttachCanvas(Canvas canvas)

    {

        _canvas = canvas;

    }



    private Canvas GetCanvas()

    {

        return _canvas
```

```
            ?? throw new InvalidOperationException("Полотно ще не
підключене.");

    }



    internal void SelectTool(DrawingTool tool)

    {

        CancelDrawing();

        _currentTool = tool;

    }



    internal bool BeginDrawing(Point position)

    {

        if (_shapeCount >= MaxShapes)

        {

            return false;

        }



        if (_currentTool == DrawingTool.Point)

        {

            Shape point = ShapeFactory.Create(

                DrawingTool.Point,

                position,
```

```
                position

            );

            AddShape(point);



            return true;

        }



        _dragStart = position;

        GetCanvas().CaptureMouse();



        return true;

    }



    internal void UpdateDrawing(Point position)

    {

        if (_dragStart is not Point start)

        {

            return;

        }



        Canvas canvas = GetCanvas();
```

```
        if (_previewShape != null)
        {
            canvas.Children.Remove(_previewShape);
        }

        Shape shape = ShapeFactory.Create(
            _currentTool,
            start,
            position
        );

        _previewShape = shape.CreateVisual(true);
        canvas.Children.Add(_previewShape);
    }

    internal void EndDrawing(Point position)
    {
        if (_dragStart is not Point start)
        {
            return;
```

```
        }


        Canvas canvas = GetCanvas();


        if (_previewShape != null)
        {
            canvas.Children.Remove(_previewShape);
            _previewShape = null;
        }


        double dx = Math.Abs(position.X - start.X);
        double dy = Math.Abs(position.Y - start.Y);


        if (dx >= 2 || dy >= 2)
        {
            Shape shape = ShapeFactory.Create(
                _currentTool,
                start,
                position
            );
```

```
            AddShape(shape);
        }

        _dragStart = null;

        if (canvas.IsMouseCaptured)
        {
            canvas.ReleaseMouseCapture();
        }
    }

    private void AddShape(Shape shape)
    {
        if (_shapeCount >= MaxShapes)
        {
            return;
        }

        _shapes[_shapeCount] = shape;
        _shapeCount++;
```

```
        GetCanvas().Children.Add(
            shape.CreateVisual(false)
        );
    }

    private void CancelDrawing()
    {
        Canvas canvas = GetCanvas();

        if (_previewShape != null)
        {
            canvas.Children.Remove(_previewShape);
            _previewShape = null;
        }

        _dragStart = null;

        if (canvas.IsMouseCaptured)
        {
            canvas.ReleaseMouseCapture();
        }
```

```
    }

}
```

ShapeFactory.cs

```
using System.Windows;

using Lab4.Models;



namespace Lab4;



internal static class ShapeFactory

{

    public static Shape Create(

        DrawingTool tool,

        Point start,

        Point end)

    {

        return tool switch

        {

            DrawingTool.Point =>

                new PointShape(start),



            DrawingTool.Line =>
```

```
                new LineShape(start, end),


            DrawingTool.Rectangle =>
                new RectangleShape(start, end),


            DrawingTool.Ellipse =>
                new EllipseShape(start, end),


            DrawingTool.LineWithCircles =>
                new LineWithCirclesShape(start, end),


            DrawingTool.Cube =>
                new CubeShape(start, end),


            _ => throw new ArgumentOutOfRangeException(nameof(tool))
        };
    }
}
```

Shape.cs

```
using System.Windows;
```

```
namespace Lab4.Models;



// абстрактний клас (неможливо створити екземпляр, призначений лише для
наслідування)

// далі override у різних дочірніх класів - поліморфізм

internal abstract class Shape

{

    protected Shape(Point start, Point end)

    {

        Start = start;

        End = end;

    }



    public Point Start { get; }

    public Point End { get; }



    public abstract UIElement CreateVisual(bool isPreview);

    public UIElement Show(bool isPreview)

    {

        return CreateVisual(isPreview);

    }

}
```

PointShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Shapes;

using System.Windows.Media;



namespace Lab4.Models;



internal class PointShape : Shape

{

    public PointShape(Point position) : base(position, position)

    {



    }



    public override UIElement CreateVisual(bool isPreview)

    {

        const double diameter = 6;



        var dot = new Ellipse

        {
```

```
            Width = diameter,
            Height = diameter,
            Fill = Brushes.Black,
            IsHitTestVisible = false
        };

        Canvas.SetLeft(dot, Start.X - diameter / 2);
        Canvas.SetTop(dot, Start.Y - diameter / 2);

        return dot;
    }
}
```

LineShape.cs

```
using System.Windows;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab4.Models;

internal class LineShape : Shape
```

```
{
    public LineShape(Point start, Point end) : base(start, end)
    {
    }

    public override UIElement CreateVisual(bool isPreview)
    {
        var line = new Line
        {
            X1 = Start.X,
            Y1 = Start.Y,
            X2 = End.X,
            Y2 = End.Y,

            Stroke = isPreview ? Brushes.Blue : Brushes.Black,
            StrokeThickness = 2,

            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,

            IsHitTestVisible = false
        };
```

```
        return line;
    }
}
```

RectangleShape.cs

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab4.Models;

internal class RectangleShape : Shape
{
    public RectangleShape(Point firstCorner, Point oppositeCorner)
    : base(firstCorner, oppositeCorner)
    {
    }

    public override UIElement CreateVisual(bool isPreview)
    {
```

```
        double left = Math.Min(Start.X, End.X);
        double top = Math.Min(Start.Y, End.Y);

        double width = Math.Abs(End.X - Start.X);
        double height = Math.Abs(End.Y - Start.Y);

        var rectangle = new Rectangle
        {
            Width = width,
            Height = height,

            Stroke = isPreview ? Brushes.Blue : Brushes.Black,
            StrokeThickness = 2,

            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,

            Fill = isPreview ? null : Brushes.White,
            IsHitTestVisible = false
        };

        Canvas.SetLeft(rectangle, left);
```

```
        Canvas.SetTop(rectangle, top);



        return rectangle;

    }

}
```

EllipseShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Media;

using System.Windows.Shapes;



namespace Lab4.Models;



internal class EllipseShape : Shape

{

    public EllipseShape(Point center, Point corner)

    : base(center, corner)

    {

    }



    public override UIElement CreateVisual(bool isPreview)
```

```
    {
        double halfWidth = Math.Abs(End.X - Start.X);
        double halfHeight = Math.Abs(End.Y - Start.Y);

        double width = halfWidth * 2;
        double height = halfHeight * 2;

        double left = Start.X - halfWidth;
        double top = Start.Y - halfHeight;

        var ellipse = new Ellipse
        {
            Width = width,
            Height = height,

            Stroke = isPreview ? Brushes.Blue : Brushes.Black,
            StrokeThickness = 2,

            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,

            Fill = null,
```

```
            IsHitTestVisible = false
        };

        Canvas.SetLeft(ellipse, left);
        Canvas.SetTop(ellipse, top);

        return ellipse;
    }
}
```

LineWithCirclesShape.cs

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab4.Models;

internal sealed class LineWithCirclesShape : Shape
{
    private const double Radius = 7;
    public LineWithCirclesShape(Point start, Point end)
```

```
        : base(start, end)
    {
    }

    public override UIElement CreateVisual(bool isPreview)
    {
        Canvas group = new Canvas
        {
            Width = Math.Max(1, Math.Max(Start.X, End.X) + Radius + 2),
            Height = Math.Max(1, Math.Max(Start.Y, End.Y) + Radius + 2),
            IsHitTestVisible = false
        };

        LineShape line = new LineShape(Start, End);
        group.Children.Add(line.Show(isPreview));

        AddCircle(group, Start, isPreview);
        AddCircle(group, End, isPreview);

        return group;
    }
```

```
    private static void AddCircle(
        Canvas group,
        Point center,
        bool isPreview)
    {
        Point corner = new Point(
            center.X + Radius,
            center.Y + Radius
        );

        EllipseShape ellipse = new EllipseShape(center, corner);
        Ellipse visual = (Ellipse)ellipse.Show(isPreview);

        visual.Fill = isPreview ? null : Brushes.LightBlue;

        group.Children.Add(visual);
    }
}
```

CubeShape.cs

```
using System.Windows;
```

```
using System.Windows.Controls;

using System.Windows.Shapes;



namespace Lab4.Models;



internal sealed class CubeShape : Shape

{

    public CubeShape(Point start, Point end)

        : base(start, end)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {

        double width = Math.Abs(End.X - Start.X);

        double height = Math.Abs(End.Y - Start.Y);



        double side = Math.Max(width, height);



        double left = Math.Min(Start.X, End.X);

        double top = Math.Min(Start.Y, End.Y);
```

```
double depth = side * 0.25;


Point[] front =
{
    new Point(left, top),
    new Point(left + side, top),
    new Point(left + side, top + side),
    new Point(left, top + side)
};


Point[] back = new Point[4];


for (int i = 0; i < 4; i++)
{
    back[i] = new Point(
        front[i].X + depth,
        front[i].Y - depth
    );
}
```

```
        Canvas group = new Canvas

        {

            Width = Math.Max(1, left + side + depth + 2),

            Height = Math.Max(1, top + side + 2),

            IsHitTestVisible = false

        };



        RectangleShape backRectangle = new RectangleShape(back[0],
back[2]);

        Rectangle backVisual = (Rectangle)backRectangle.Show(isPreview);

        backVisual.Fill = null;



        group.Children.Add(backVisual);



        //  з'єднувальні лінії

        for (int i = 0; i < 4; i++)

        {

            LineShape edge = new LineShape(

                front[i],

                back[i]

            );
```

```
            group.Children.Add(edge.Show(isPreview));

        }



        RectangleShape frontRectangle = new RectangleShape(front[0],
front[2]);

        Rectangle frontVisual = (Rectangle)frontRectangle.Show(isPreview);

        frontVisual.Fill = null;



        group.Children.Add(frontVisual);

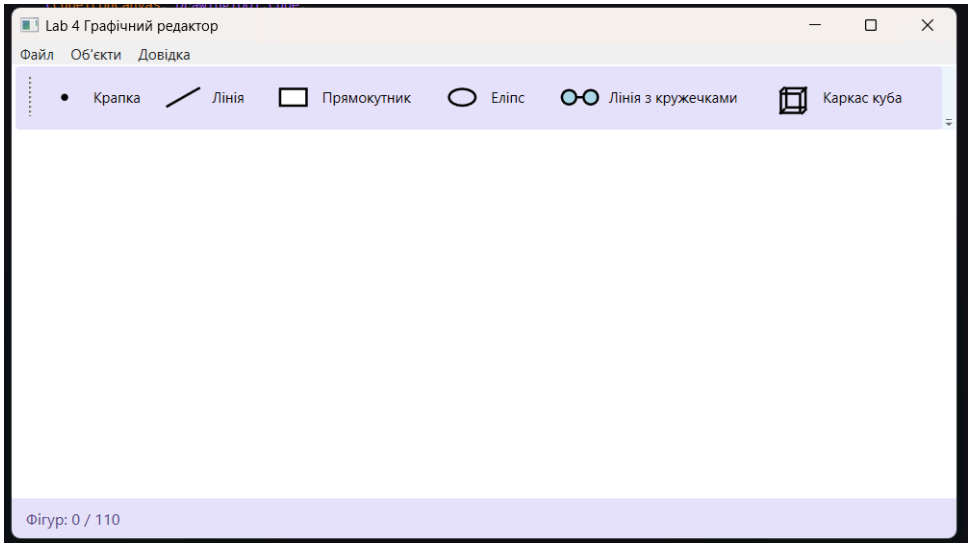


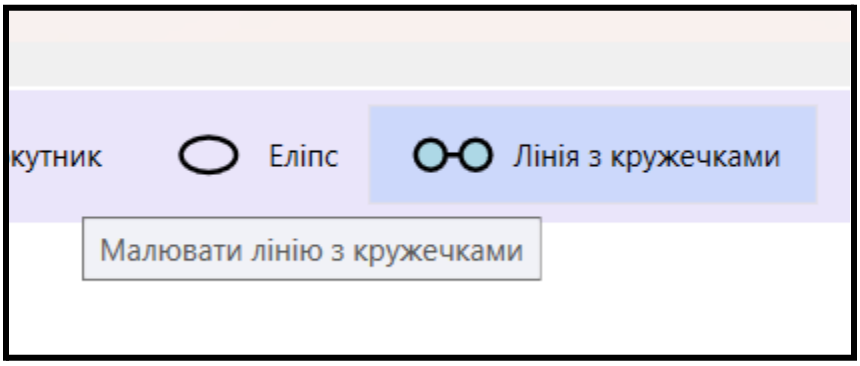
        return group;

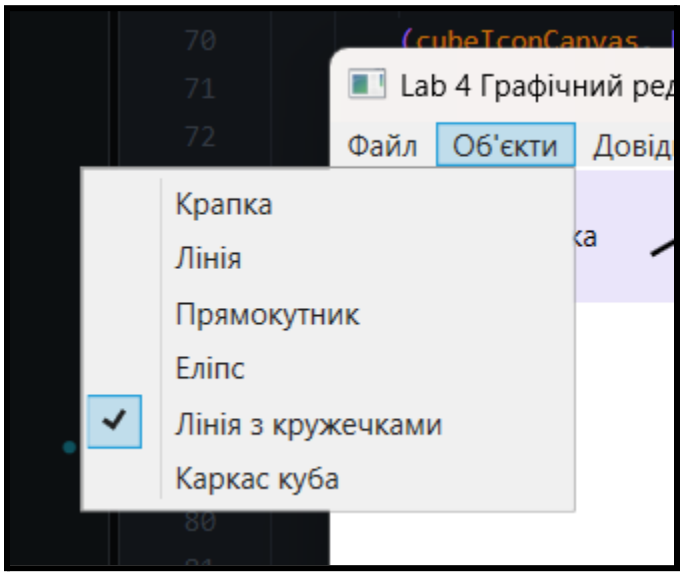
    }

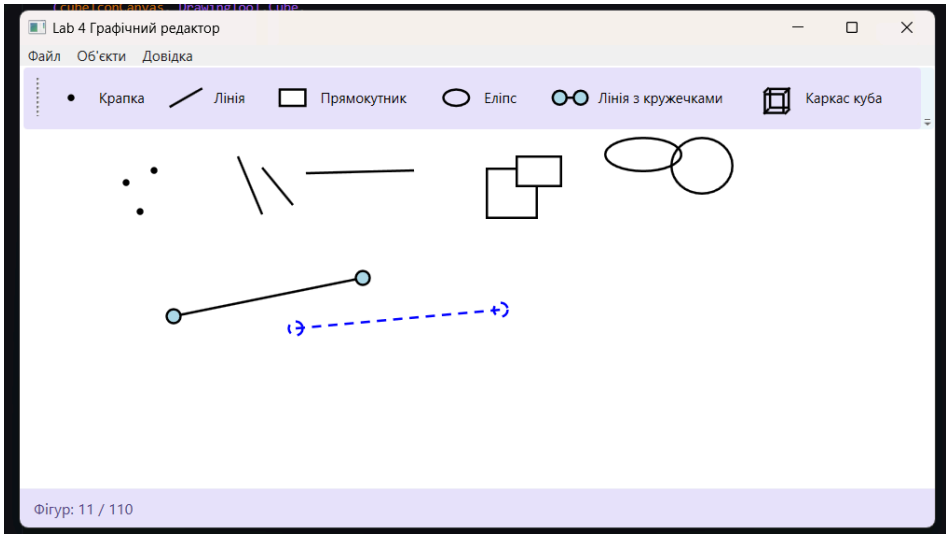
}
```

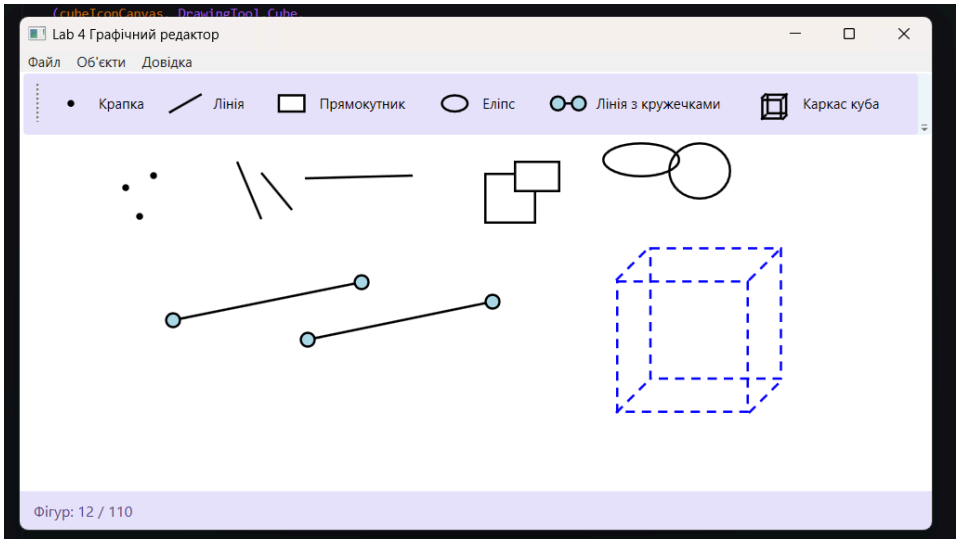
Скріншоти

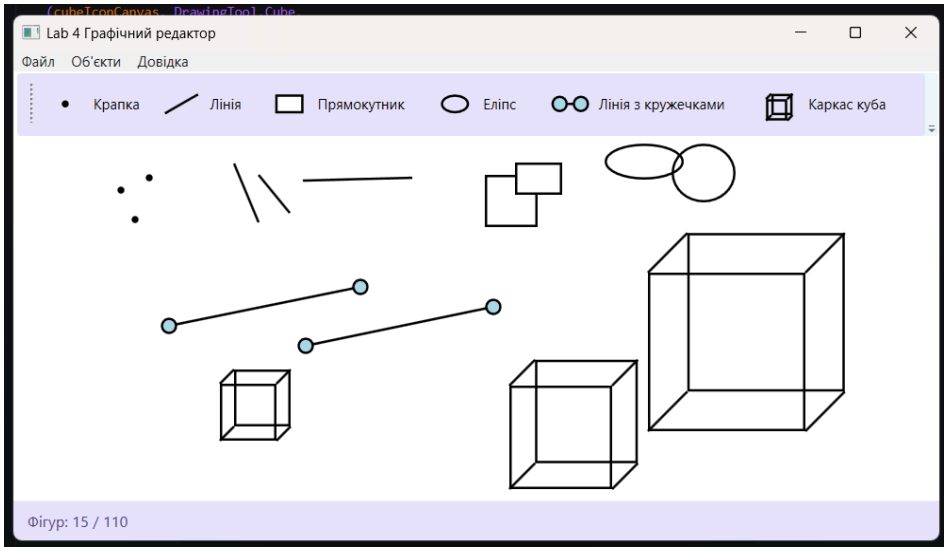

Lab 4 Графічний редактор
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба
Фігур: 0 / 110


кутник
Еліпс
Лінія з кружечками
Малювати лінію з кружечками


Lab 4 Графічний ред
Файл
Об'єкти
Довід
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба


Lab 4 Графічний редактор
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба
Фігур: 11 / 110


Lab 4 Графічний редактор
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба
Фігур: 12 / 110


Lab 4 Графічний редактор
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба
Фігур: 15 / 110

Діаграма залежностей файлів та модулів

```
Lab4.csproj
├── App.xaml
├── App.xaml.cs
├── MainWindow.xaml
├── MainWindow.xaml.cs
├── MyEditor.cs
├── ShapeFactory.cs
├── DrawingTool.cs
└── Models/
    ├── Shape.cs
    ├── PointShape.cs
    ├── LineShape.cs
    ├── RectangleShape.cs
    ├── EllipseShape.cs
    ├── LineWithCirclesShape.cs
    └── CubeShape.cs
```

```
App.xaml
├── App.xaml.cs
└── MainWindow.xaml (StartupUri)
```

```
App.xaml.cs
├── System.Windows
│   └── Application
└── MyEditor.cs
    └── App.Editor (static readonly)
```

```
MainWindow.xaml
```

```
├── MainWindow.xaml.cs
└── System.Windows.Controls
    ├── DockPanel
    ├── Menu
    ├── MenuItem
    ├── ToolBar
    ├── Button
    │   └── ToolTip
    ├── Canvas
    ├── Border
    └── TextBlock
```

```
MainWindow.xaml.cs
├── System.Windows
├── System.Windows.Controls
├── System.Windows.Input
├── App.xaml.cs
│   └── App.Editor
│       └── MyEditor.cs
├── DrawingTool.cs
│   └── DrawingTool (enum)
└── Методи обробки подій
    ├── OnNotify()
    ├── SelectTool()
    ├── Point_Click()
    ├── Line_Click()
    ├── Rectangle_Click()
    ├── Ellipse_Click()
    ├── LineWithCircles_Click()
    ├── Cube_Click()
    ├── DrawingCanvas_MouseLeftButtonDown()
```

```
    ├── DrawingCanvas_MouseMove()
    ├── DrawingCanvas_MouseLeftButtonUp()
    └── UpdateShapeCount()


MyEditor.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── DrawingTool.cs
├── ShapeFactory.cs
│   └── ShapeFactory.Create()
├── Models/
│   └── Shape.cs
│       └── Shape (abstract class)
└── Методи керування редактором
    ├── AttachCanvas()
    ├── GetCanvas()
    ├── SelectTool()
    ├── BeginDrawing()
    ├── UpdateDrawing()
    ├── EndDrawing()
    ├── AddShape()
    └── CancelDrawing()


ShapeFactory.cs
├── DrawingTool.cs
│   └── DrawingTool (enum)
├── System.Windows
│   └── Point
└── Models/
```

```
    ├── Shape.cs
    │   └── Shape (abstract class)
    ├── PointShape.cs
    ├── LineShape.cs
    ├── RectangleShape.cs
    ├── EllipseShape.cs
    ├── LineWithCirclesShape.cs
    └── CubeShape.cs


DrawingTool.cs
└── DrawingTool (enum)
    ├── Point
    ├── Line
    ├── Rectangle
    ├── Ellipse
    ├── LineWithCircles
    └── Cube


Models/Shape.cs
└── System.Windows
    ├── Point
    └── UIElement


Models/PointShape.cs
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
```

```
│   └── Brushes
└── System.Windows.Shapes
    └── Ellipse
```

```
Models/LineShape.cs
├── Shape.cs
├── System.Windows
├── System.Windows.Media
│   ├── Brushes
│   └── DoubleCollection
└── System.Windows.Shapes
    └── Line
```

```
Models/RectangleShape.cs
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
│   ├── Brushes
│   └── DoubleCollection
└── System.Windows.Shapes
    └── Rectangle
```

```
Models/EllipseShape.cs
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
```

```
├── System.Windows.Media
│   ├── Brushes
│   └── DoubleCollection
└── System.Windows.Shapes
    └── Ellipse
```

Models/LineWithCirclesShape.cs

```
├── Shape.cs
├── LineShape.cs
│   └── LineShape.Show()
├── EllipseShape.cs
│   └── EllipseShape.Show()
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
│   └── Brushes
└── System.Windows.Shapes
    └── Ellipse
```

Models/CubeShape.cs

```
├── Shape.cs
├── RectangleShape.cs
│   └── RectangleShape.Show()
├── LineShape.cs
│   └── LineShape.Show()
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
└── System.Windows.Shapes
```

```
    └── Rectangle
```

Діаграма успадкування класів

```
Shape (abstract class)
|
├── PointShape
|   └── CreateVisual(bool isPreview)
|
├── LineShape
|   └── CreateVisual(bool isPreview)
|
├── RectangleShape
|   └── CreateVisual(bool isPreview)
|
├── EllipseShape
|   └── CreateVisual(bool isPreview)
|
├── LineWithCirclesShape
|   ├── CreateVisual(bool isPreview)
|   └── Композиція:
|       ├── LineShape
|       └── EllipseShape
|
└── CubeShape
    ├── CreateVisual(bool isPreview)
    └── Композиція:
        ├── RectangleShape
        └── LineShape
```

Висновки:

Під час виконання лабораторної роботи було вдосконалено структуру графічного редактора мовою C# із використанням WPF. Реалізовано класи MyEditor та ShapeFactory, а також додано два нові типи геометричних фігур із застосуванням композиції. У результаті закріплено навички об'єктно-орієнтованого проєктування, повторного використання коду та створення гнучкої архітектури, що спрощує розширення функціональності програми.